FORMACIÓN PROFESIONAL · GRADO SUPERIOR

DAM2

UT2: Manejo de Conectores (JDBC y MySQL)

Manual Técnico y Guía de Persistencia Empresarial: Arquitectura JDBC, Transacciones ACID, Connection Pools (HikariCP) y Patrones de Diseño (Singleton, DTO y DAO)

MÓDULO
Acceso a Datos (Código oficial: 0486)

UNIDAD DIDÁCTICA
UD02 / UT2 (13 Horas Lectivas Presenciales)

RESULTADOS DE APRENDIZAJE
RA2 (CE2.a – CE2.k) · Ponderación: 20%

CURSO ACADÉMICO
Curso 2026-2027 · CIFP Avilés · Sergio Capdevila Díez

📑 ÍNDICE DE CONTENIDOS

1. Introducción y Contexto Profesional
2. Fundamentos Comunes
   - 2.1. Arquitectura JDBC y el Desfase Objeto-Relacional
   - 2.2. Gestores de Bases de Datos: Embebidos e Independientes
   - 2.3. Arquitectura JDBC, Ciclo de Conexión y Liberación de Recursos
3. Bloque 1 · Conexión y Uso de Base de Datos Relacional con JDBC (MySQL)
   - 3.1. Driver JDBC de MySQL y Conexión con DriverManager
   - 3.2. Operaciones CRUD con PreparedStatement
   - 3.3. Consultas con ResultSet y Tratamiento de Nulos
   - 3.4. Subprogramas en el Motor: CallableStatement y Procedimientos Almacenados
   - 3.5. Control de Transacciones ACID y Savepoints
   - 3.6. Definición DDL e Introspección de Metadatos
4. Bloque 2 · Patrones de Diseño y Connection Pools en el Acceso a Datos
   - 4.1. Pools de Conexiones y Configuración con HikariCP
   - 4.2. Patrones Singleton y DTO con Java Records · ¿por qué patrones? · Singleton · DTO
   - 4.3. Patrón DAO Desacoplado con Inyección de DataSource · GenericDAO · recorrido de una llamada · resumen
5. Integración: Backend Bancario Transaccional con DAO y Pool
   - 5.1. Backend Bancario Transaccional con DAO y Pool
6. Visión de Conjunto y Buenas Prácticas de Ingeniería
7. Resumen y Conceptos que Debes Dominar
8. Glosario Técnico y Cuestionario de Repaso

Anexo: Trazabilidad Curricular Oficial

1. Introducción y Contexto Profesional

En el desarrollo de software empresarial contemporáneo, los datos transaccionales críticos (cuentas bancarias, historiales médicos, stock de inventarios en tiempo real y facturación) se almacenan de forma masiva en **Sistemas Gestores de Bases de Datos Relacionales (RDBMS)**. Motores industriales como **MySQL**, **PostgreSQL**, **Oracle Database** o **SQL Server** garantizan propiedades ACID estrictas que aseguran la consistencia y durabilidad del dato frente a caídas eléctricas o fallos concurrentes.

El estándar fundamental del ecosistema Java para interactuar de forma programática con estos motores es **JDBC (Java Database Connectivity)**. Aunque en módulos avanzados o en proyectos modernos se empleen capas de abstracción superior (como frameworks ORM tipo Hibernate o JPA), JDBC constituye la **infraestructura subyacente obligatoria** que todo ingeniero de software debe dominar para:

- Comprender exactamente qué comandos SQL se envían a través de la red TCP al servidor de base de datos.
- Optimizar consultas críticas donde los generadores automáticos de ORM producen consultas ineficientes o cuellos de botella.
- Controlar transacciones distribuidas y lotes masivos de inserción (*batch processing*) con coste mínimo de latencia.
- Configurar piscinas de conexiones físicas de alto rendimiento (*connection pools*) para entornos cloud y microservicios concurrentes.

> 💡 **Alineación Curricular Oficial**
>
> Esta unidad desarrolla íntegramente el **Resultado de Aprendizaje 2 (RA2)** del módulo profesional de Acceso a Datos (Real Decreto 405/2023):
>
> *«Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión.»*
>
> A lo largo de los temas se cubren exhaustivamente sus 11 criterios de evaluación: desde las ventajas e inconvenientes de los conectores (**CE2.a**), bases embebidas vs cliente-servidor (**CE2.b**), uso del driver idóneo (**CE2.c**), conexión (**CE2.d**), DDL (**CE2.e**), DML (**CE2.f**), mapeo de resultados (**CE2.g**), consultas (**CE2.h**), liberación de recursos (**CE2.i**), transacciones ACID (**CE2.j**) y procedimientos almacenados (**CE2.k**).

> 🏭 **Requisitos Previos y Metodología de Aula**
>
> Para abordar con éxito esta unidad de trabajo, necesitas tener asimilados los conceptos fundamentales de Programación Orientada a Objetos en Java (clases, interfaces, herencia, colecciones y manejo de excepciones) y la gestión de recursos vista en la **UT1** mediante la sentencia `try-with-resources` y la interfaz `AutoCloseable`.
>
> El aprendizaje se estructura en torno a una **metodología acompañada**: tras la explicación teórica de cada apartado, abrirás Eclipse IDE en el aula de informática y resolverás de forma guiada la actividad correspondiente de tu Cuaderno de Actividades de Aula sobre un servidor real **MySQL**.

2. Fundamentos Comunes

Antes de escribir código Java contra una base de datos específica, es imprescindible comprender la arquitectura conceptual sobre la que descansa el estándar JDBC y la naturaleza del motor de persistencia con el que dialogamos.

2.1. Arquitectura JDBC y el Desfase Objeto-Relacional

En las aplicaciones empresariales conviven dos paradigmas conceptualmente incompatibles:

1. **El Paradigma Orientado a Objetos (Java):** Se basa en grafos de objetos interconectados en memoria RAM, con estado, identidad (referencia en memoria), encapsulamiento, polimorfismo y herencia.
2. **El Paradigma Relacional (SQL / RDBMS):** Se basa en el álgebra relacional, donde los datos se organizan en relaciones matemáticas bidimensionales (tablas formadas por filas y columnas atómicas), vinculadas exclusivamente mediante valores escalares coincidentes (claves foráneas) y restricciones de integridad.

Esta fricción conceptual se denomina en ingeniería de software como **Desfase Objeto-Relacional** (*Object-Relational Impedance Mismatch*). Provoca que los tipos de datos no coincidan de forma directa (por ejemplo, tipos de fecha, números decimales de precisión arbitraria como `DECIMAL` frente a `BigDecimal`, o el manejo del valor `NULL`).

> 💡 **¿Qué es un Conector JDBC?**
>
> **JDBC (Java Database Connectivity)** es una API estándar de Java (ubicada en los paquetes `java.sql` y `javax.sql`) que define un conjunto de interfaces universales independientes del proveedor. El **conector** (o *driver* JDBC) es la implementación concreta de esas interfaces proporcionada por el fabricante del motor de base de datos (MySQL, PostgreSQL, Oracle) que traduce las llamadas universales de Java al protocolo de red binario nativo del servidor.

ASPECTO	ACCESO DIRECTO CON CONECTORES JDBC	MAPEO OBJETO-RELACIONAL (ORM / JPA)
Rendimiento	**Máximo y predecible.** Sin sobrecarga de reflexión ni cachés intermedias ocultas.	Ligera sobrecarga por serialización y gestión de ciclo de vida de entidades.
Control del SQL	**Total.** El desarrollador escribe y optimiza cada sentencia SQL a medida.	Automático. A menudo genera consultas complejas o subóptimas (problema N+1).
Productividad	Menor para CRUDs triviales (requiere código repetitivo de mapeo manual).	Muy alta para operaciones transaccionales sencillas de dominio.
Portabilidad	Depende de ceñirse a SQL estándar; sentencias propietarias (por ejemplo, la sintaxis de los procedimientos almacenados) ligan al motor.	Alta: el dialecto ORM traduce entidades a múltiples motores relacionales.

2.2. Gestores de Bases de Datos: Embebidos e Independientes

A la hora de diseñar una solución de persistencia, el ingeniero debe elegir la tipología de motor adecuada según el contexto de despliegue:

- **Gestores de Bases de Datos Embebidos (In-Process):** Ejemplos destacados son **H2 Database**, **SQLite** o **Apache Derby**. El motor de la base de datos se ejecuta *dentro de la misma Máquina Virtual de Java (JVM)* que la aplicación. No requiere instalar un servicio en el sistema operativo ni abrir puertos de red TCP. Son ideales para pruebas unitarias automatizadas (base en memoria RAM `jdbc:h2:mem:testdb`), herramientas de línea de comandos monousuario o aplicaciones móviles. Su principal limitación es que no toleran accesos concurrentes de cientos de usuarios distribuidos ni ofrecen capacidades de clustering empresarial.
- **Gestores de Bases de Datos Independientes (Client-Server):** Ejemplos industriales son **MySQL**, **PostgreSQL** u **Oracle Database**. El motor opera como un servicio autónomo o cluster en servidores dedicados. Escucha peticiones en un socket TCP/IP (por ejemplo, el puerto 3306 en MySQL), atiende cada conexión con un hilo propio, mantiene en memoria cachés compartidas (el *buffer pool* de InnoDB) y registra los cambios en logs transaccionales (*redo log*) para poder recuperarse ante una caída. Permiten concurrencia masiva, control de acceso por usuario y base de datos, y transacciones ACID.

2.3. Arquitectura JDBC, Ciclo de Conexión y Liberación de Recursos

La arquitectura de JDBC se divide en cuatro capas fundamentales:

1. **Aplicación Java:** Invoca métodos estándar de las interfaces `Connection`, `PreparedStatement` y `ResultSet`.
2. **JDBC Driver Manager (`DriverManager`):** Localiza y carga el driver adecuado en función de la URL de conexión solicitada.

3. **Driver JDBC del Fabricante:** Convierte las órdenes JDBC a tramas de red binarias nativas (en MySQL, el protocolo cliente/servidor de MySQL).
4. **Servidor de Base de Datos:** Procesa el SQL, ejecuta el plan en su motor relacional y retorna los cursores de datos.

> ⚠️ **Regla Crítica de Ingeniería: La Liberación Obligatoria de Recursos**
>
> Una conexión de base de datos no es solo memoria en Java: **representa un socket de red TCP activo y una sesión reservada con memoria y un hilo dedicado en el servidor MySQL**. Si una aplicación crea conexiones o sentencias sin cerrarlas, se produce una *fuga de conexiones (connection leak)* que saturará la base de datos hasta agotar el máximo de conexiones permitido por el servidor (`max_connections`) y provocar el error `1040: Too many connections` , que deniega el servicio por completo.
>
> En Java moderno es **estrictamente obligatorio** utilizar la construcción `try-with-resources` para garantizar el cierre automático de `Connection` , `Statement` / `PreparedStatement` y `ResultSet` , incluso si ocurre una excepción inesperada.

> 🧠 **Comprueba que sabes (Fundamentos)**
>
> 1. ¿Por qué se produce el desfase objeto-relacional y qué problemas causa en el mapeo de tipos?
> 2. ¿En qué escenarios utilizarías una base de datos embebida como H2 y cuándo un motor independiente como MySQL?
> 3. Menciona las tres interfaces centrales de JDBC que implementan `AutoCloseable` y deben liberarse siempre de forma determinista.

3. Bloque 1 · Conexión y Uso de Base de Datos Relacional con JDBC (MySQL)

En este bloque aprenderás a trabajar de forma directa contra un servidor **MySQL**, dominando la parametrización de consultas, el manejo del cursor de resultados, las transacciones manuales y los subprogramas en el motor.

3.1. Driver JDBC de MySQL y Conexión con DriverManager

Para conectar Java 21 con MySQL 8, se emplea el driver oficial **MySQL Connector/J** (`mysql-connector-j`). Se trata de un driver de Tipo 4 (100% Java puro), que no requiere la instalación de software cliente nativo de MySQL en la máquina local.

La incorporación del conector se realiza mediante la siguiente dependencia de Maven en el archivo `pom.xml` :

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.4.0</version>
</dependency>
```

La estructura de la URL JDBC de MySQL sigue el estándar:

```
jdbc:mysql://<host>:<puerto>/<base_de_datos>?<opciones>
// Ejemplo para el servidor MySQL local del aula:
jdbc:mysql://localhost:3306/dam2_db?useSSL=false&allowPublicKeyRetrieval=true
```

> 💻 **Actividad 2.1 del cuaderno de aula**
>
> **Conexión Inicial y Diagnóstico con MySQL:** En este reto de aula incorporarás la dependencia `mysql-connector-j` en Maven, establecerás una conexión con `DriverManager.getConnection()` e inspeccionarás los metadatos de versión del producto y catálogo de MySQL.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.1

3.2. Operaciones CRUD con PreparedStatement

Casi todo lo que una aplicación hace con una tabla se reduce a cuatro operaciones, el **CRUD**: *Create* (alta, `INSERT`), *Read* (consulta, `SELECT`), *Update* (modificación, `UPDATE`) y *Delete* (baja, `DELETE`). En JDBC las cuatro se programan igual: se escribe la sentencia con marcadores `?` , se prepara con `conn.prepareStatement(sql)` , se da valor a cada `?` con un método `setXxx` y se ejecuta. Lo único que cambia es **el método de ejecución y lo que devuelve**.

OPERACIÓN	SENTENCIA SQL	MÉTODO	DEVUELVE	CÓMO SE INTERPRETA
C · Alta	`INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)`	`executeUpdate()`	`int`	Filas insertadas (1). La clave nueva se lee con `getGeneratedKeys()`.
R · Consulta	`SELECT ... FROM productos WHERE id_producto = ?`	`executeQuery()`	`ResultSet`	Las filas encontradas. Si `rs.next()` devuelve `false` a la primera, no hay ninguna.
U · Modificación	`UPDATE productos SET precio = ?, stock = ? WHERE id_producto = ?`	`executeUpdate()`	`int`	Filas modificadas: `1` si existía, `0` si el `WHERE` no encontró nada.
D · Baja	`DELETE FROM productos WHERE id_producto = ?`	`executeUpdate()`	`int`	Filas borradas: `1` si existía, `0` si no.

> 💡 **Por qué siempre `PreparedStatement` y nunca concatenar**
>
> Con `Statement` y concatenación, el valor que escribe el usuario pasa a formar parte del texto SQL: si en `"DELETE FROM productos WHERE id_producto = " + entrada` alguien escribe `1 OR 1=1`, se borra la tabla entera (**inyección SQL**). Con `PreparedStatement` la sentencia y los valores viajan por separado: el valor de cada `?` se trata siempre como un dato, nunca como código SQL. Además, los métodos `setString()`, `setInt()`, `setLong()` o `setBigDecimal()` convierten cada valor al tipo SQL correcto y se ocupan de las comillas y los caracteres especiales.
>
> Los `?` se numeran **desde 1**, en el orden en que aparecen en la sentencia. Para importes se usa siempre `BigDecimal` (creado desde texto: `new BigDecimal("74.50")`), nunca `double`.

C · Alta con recuperación de la clave generada. Cuando la tabla tiene una clave `BIGINT AUTO_INCREMENT`, se pide a MySQL que devuelva el valor que ha asignado indicando `Statement.RETURN_GENERATED_KEYS` al preparar la sentencia:

```
static long insertarProducto(Connection conn, String nombre, BigDecimal precio, int stock) throws
SQLException {
    String sql = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
    try (PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, nombre);
        ps.setBigDecimal(2, precio);
        ps.setInt(3, stock);
```

```
        int filas = ps.executeUpdate();             // 1 fila insertada
        try (ResultSet claves = ps.getGeneratedKeys()) {
            claves.next();
            return claves.getLong(1);               // ID asignado por MySQL
        }
    }
}
```

R · Consulta. Un `SELECT` se ejecuta con `executeQuery()`, que devuelve un `ResultSet`: las filas del resultado y un cursor colocado *antes* de la primera. Cada llamada a `rs.next()` avanza a la fila siguiente y devuelve `false` cuando ya no quedan; sobre la fila actual se leen las columnas por su nombre con `getLong()`, `getString()`, `getBigDecimal()` o `getInt()`. Si se busca por clave primaria habrá como mucho una fila, así que basta un `if`; para listar se usa un `while`:

```
// Buscar por clave primaria: 0 o 1 fila -> if
String sql = "SELECT id_producto, nombre, precio, stock FROM productos WHERE id_producto = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setLong(1, id);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            System.out.println(rs.getString("nombre") + " | " + rs.getBigDecimal("precio") + " €");
        } else {
            System.out.println("No existe ningún producto con ID " + id);
        }
    }
}

// Listar toda la tabla: muchas filas -> while
try (PreparedStatement ps = conn.prepareStatement("SELECT id_producto, nombre FROM productos ORDER BY
id_producto");
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getLong("id_producto") + " - " + rs.getString("nombre"));
    }
}
```

U y D · Modificación y baja. Se ejecutan con `executeUpdate()` y lo importante es su valor de retorno: el número de filas afectadas. Un `0` no es un error de Java, sino la forma de saber que el `WHERE` no encontró ninguna fila (por ejemplo, porque el ID no existe):

```
static int actualizarPrecioYStock(Connection conn, long id, BigDecimal precio, int stock) throws SQLException
{
    String sql = "UPDATE productos SET precio = ?, stock = ? WHERE id_producto = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setBigDecimal(1, precio);
        ps.setInt(2, stock);
        ps.setLong(3, id);              // tercer ?: el orden de los set sigue el de la SQL
        return ps.executeUpdate();      // 1 = modificado, 0 = no existe
    }
}

static int eliminarProducto(Connection conn, long id) throws SQLException {
    try (PreparedStatement ps = conn.prepareStatement("DELETE FROM productos WHERE id_producto = ?")) {
```

```
            ps.setLong(1, id);
            return ps.executeUpdate();       // 1 = borrado, 0 = no existe
        }
    }
```

> ⚠️ **Errores típicos en el CRUD**
>
> - Usar `executeQuery()` con un `INSERT`, `UPDATE` o `DELETE`, o `executeUpdate()` con un `SELECT`: ambos lanzan `SQLException`.
> - Numerar los parámetros desde 0 (`setString(0, ...)`) o darles valor en un orden distinto al de los `?`.
> - Leer columnas sin haber llamado antes a `rs.next()`: el cursor todavía no está sobre ninguna fila.
> - Escribir un `UPDATE` o un `DELETE` sin `WHERE`: afecta a **todas** las filas de la tabla.

> 💻 **Actividad 2.2 del cuaderno de aula**
>
> **CRUD Completo del Catálogo con PreparedStatement:** programa un método por operación (alta con clave generada, búsqueda por ID, listado, modificación de precio y stock, y baja) y comprueba el caso de un ID inexistente, que devuelve 0 filas afectadas.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.2

3.3. Consultas con ResultSet y Tratamiento de Nulos

En el apartado anterior se usó el `ResultSet` para leer filas sencillas; ahora se estudia con más detalle. La ejecución de consultas `SELECT` se realiza mediante el método `executeQuery()`, el cual retorna una instancia de `ResultSet`. El `ResultSet` actúa como un **cursor tabular** posicionado inicialmente justo antes de la primera fila de datos. El método `next()` avanza el cursor fila a fila devolviendo `true` mientras existan registros pendientes de lectura.

> **⚠️ La Trampa de los Tipos Primitivos y el Valor NULL en JDBC**
>
> En SQL, cualquier columna que no tenga restricción `NOT NULL` puede albergar el valor `NULL`. Sin embargo, en Java los tipos primitivos (`int`, `double`, `long`, `boolean`) no pueden representar la ausencia de valor. Si lees una columna con valor SQL `NULL` mediante `rs.getInt("descuento")`, **el driver JDBC retornará silenciosamente el número 0**, lo cual provocará graves fallos de lógica de negocio (no es lo mismo un 0% de descuento que la ausencia de descuento aplicado).
>
> Existen dos soluciones estándar en la industria:
>
> 1. **Patrón `rs.wasNull()`:** Inmediatamente después de llamar al getter primitivo, se invoca `if (rs.wasNull()) { valor = null; }`.
> 2. **Método moderno de JDBC 4.2 `rs.getObject(col, Class<T>)`:** Permite solicitar directamente el wrapper de objeto: `Double desc = rs.getObject("descuento", Double.class);`. Si en base de datos era NULL, devolverá la referencia `null` legítima.

> **💻 Actividad 2.3 del cuaderno de aula**
>
> **Navegación de ResultSet y Tratamiento Estricto de NULLs:** Desarrolla consultas filtradas dinámicamente, extrae fechas tipadas con `LocalDate` y gestiona columnas numéricas con valores nulos empleando tanto `wasNull()` como `rs.getObject()`.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.3

3.4. Subprogramas en el Motor: CallableStatement y Procedimientos Almacenados

En aplicaciones de alto rendimiento o banca corporativa, parte de la lógica crítica de negocio reside compilada dentro del propio motor de base de datos en forma de procedimientos o funciones almacenadas (en MySQL, con sentencias SQL dentro de un bloque `BEGIN ... END`). Esto reduce radicalmente el tráfico de red TCP al evitar múltiples idas y venidas de datos entre el servidor de aplicaciones y la base de datos.

La invocación de estos subprogramas desde Java se realiza mediante la interfaz `CallableStatement`:

```
// Sintaxis estándar JDBC para procedimientos con parámetros IN y OUT:
String llamada = "{ call sp_resumen_producto(?, ?, ?) }";

try (CallableStatement cs = conn.prepareCall(llamada)) {
    // 1. Configurar parámetros de entrada (IN)
    cs.setLong(1, 101L);

    // 2. Registrar los tipos de datos de salida (OUT) en el diccionario JDBC
    cs.registerOutParameter(2, java.sql.Types.DECIMAL);
    cs.registerOutParameter(3, java.sql.Types.VARCHAR);

    // 3. Ejecutar en el servidor MySQL
```

```
    cs.execute();

    // 4. Leer los valores devueltos por el procedimiento
    BigDecimal precio = cs.getBigDecimal(2);
    String estado = cs.getString(3);
    System.out.println("Precio: " + precio + " · Estado: " + estado);
}
```

> 💻 **Actividad 2.4 del cuaderno de aula**
>
> **Invocación de Procedimientos Almacenados:** Crea un procedimiento almacenado en MySQL con parámetros IN y OUT, e invócalo desde Java con `CallableStatement` registrando los tipos SQL con `java.sql.Types`.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.4

3.5. Control de Transacciones ACID y Savepoints

Por defecto, toda conexión JDBC opera en modo **auto-commit activado** (`autoCommit = true`): cada sentencia `executeUpdate()` individual se confirma de forma automática e irrevocable en el disco del motor. Sin embargo, en operaciones de negocio compuestas (por ejemplo, transferir dinero de la Cuenta A a la Cuenta B), se requiere que ambas operaciones triunfen juntas o fracasen por completo sin alterar los saldos (principio de **Atomicidad**).

El patrón profesional de control transaccional manual en JDBC se estructura así:

```
try (Connection conn = dataSource.getConnection()) {
    // 1. Desactivar auto-commit para iniciar la frontera transaccional
    conn.setAutoCommit(false);

    try {
        // Operación 1: Retirar fondos
        debitarCuenta(conn, ibanOrigen, cantidad);

        // Operación intermedia: Savepoint opcional para operaciones no críticas
        Savepoint spAuditoria = conn.setSavepoint("SP_AUDITORIA");

        // Operación 2: Abonar fondos
        acreditarCuenta(conn, ibanDestino, cantidad);

        // 2. Si todo fue correcto, confirmar definitivamente en el motor
        conn.commit();
        System.out.println("Transferencia confirmada con éxito.");

    } catch (Exception ex) {
        // 3. Ante cualquier fallo de negocio o SQL, deshacer todas las modificaciones
        conn.rollback();
        System.err.println("Transacción abortada. Rollback ejecutado: " + ex.getMessage());
        throw ex;
    } finally {
        // 4. Restaurar el estado predeterminado de la conexión antes de devolverla
```

```
            conn.setAutoCommit(true);
        }
    }
```

> ⚠️ **En MySQL, las transacciones exigen tablas InnoDB**
>
> Solo el motor de almacenamiento **InnoDB** (el de por defecto en MySQL 8) soporta `commit()`, `rollback()` y `Savepoint`. En una tabla MyISAM, el `rollback()` no deshace nada. Además, las sentencias DDL (`CREATE TABLE`, `ALTER TABLE`...) provocan un **commit implícito** de la transacción en curso. El nivel de aislamiento por defecto de InnoDB es `REPEATABLE READ`.

> 💻 **Actividad 2.5 del cuaderno de aula**
>
> **Gestión Transaccional ACID Bancaria y Savepoints:** Desarrolla una transferencia entre cuentas bancarias con control transaccional manual, simulando caídas de negocio para certificar que el `rollback()` previene pérdidas de fondos, y usa un `Savepoint` para anular solo la comisión de una transferencia.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.5

3.6. Definición DDL e Introspección de Metadatos

Además de manipular datos, JDBC permite crear, alterar y destruir estructuras de tablas (DDL) y examinar dinámicamente la configuración del motor sin conocer el esquema de antemano mediante dos interfaces de metadatos:

- `DatabaseMetaData` **(a través de** `conn.getMetaData()` **):** Ofrece información global sobre las capacidades del gestor (soporte transaccional, versión del producto) y permite explorar el catálogo (tablas del usuario con `getTables()`, columnas de una tabla con `getColumns()` y claves primarias con `getPrimaryKeys()`).
- `ResultSetMetaData` **(a través de** `rs.getMetaData()` **):** Describe la estructura de una consulta `SELECT` concreta en tiempo de ejecución: número de columnas proyectadas (`getColumnCount()`), nombres o alias de columnas (`getColumnLabel()`), tipos de datos nativos SQL (`getColumnTypeName()`) y si admiten nulos (`isNullable()`).

> 💡 **Esquema y catálogo en MySQL**
>
> En MySQL «esquema» y «base de datos» son sinónimos y JDBC los expone como **catálogo**. Por eso, para listar las tablas se usa `dmd.getTables(conn.getCatalog(), null, "%", new String[] {"TABLE"})`: el nombre de la base de datos va en el primer parámetro y el de esquema se deja a `null`.

💻 Actividad 2.6 del cuaderno de aula

Introspección Dinámica con DatabaseMetaData y ResultSetMetaData: Construye un visor genérico de esquemas que consulte las tablas de tu base de datos MySQL y proyecte en consola sus columnas, tipos de datos y nulabilidad.

📂 Enlace directo: Ver enunciado de la Actividad 2.6

🧠 Comprueba que sabes (Bloque 1)

1. ¿Por qué `PreparedStatement` previene de forma matemática la Inyección SQL frente a concatenar cadenas en `Statement`?
2. En un CRUD con JDBC, ¿qué método ejecuta un `SELECT` y cuál un `INSERT`, `UPDATE` o `DELETE`? ¿Qué significa que `executeUpdate()` devuelva `0`?
3. ¿Qué problema presenta `rs.getInt("columna")` si el valor en la base de datos es SQL `NULL` y cómo se subsana?
4. Explica el ciclo canónico de una transacción en JDBC: qué método inicia el bloque atómico, qué método lo confirma y qué método lo deshace.

✅ Ya puedes hacer

Con las bases operativas de JDBC, sentencias parametrizadas, gestión de cursores, transacciones y metadatos superadas, dispones de todos los conocimientos necesarios para resolver:

- **AD-UT2-B1-Ejercicios-JDBC**: CLI Interactivo de Inventario con PreparedStatement y Transacciones Bancarias con Auditoría de Fallos y Savepoints.

4. Bloque 2 · Patrones de Diseño y Connection Pools en el Acceso a Datos

En el software profesional, el código SQL jamás se dispersa de manera caótica dentro de los controladores o interfaces de usuario. En este bloque aprenderás a aplicar los patrones de persistencia estándar de la industria y a optimizar el rendimiento concurrente mediante pools de conexiones.

4.1. Pools de Conexiones y Configuración con HikariCP

Abrir una conexión física contra MySQL mediante `DriverManager.getConnection()` es una de las operaciones más lentas y costosas en informática empresarial: requiere establecer el handshake TCP, autenticar credenciales criptográficas, asignar memoria de sesión en el servidor y crear un hilo de conexión en el servidor MySQL. Este proceso suele tomar decenas de milisegundos (entre **10 y 100 ms** según la autenticación y el cifrado) por cada llamada.

Si una aplicación web con 500 peticiones concurrentes abriera y cerrara conexiones con `DriverManager` en cada click de usuario, el servidor colapsaría inmediatamente. La solución industrial consiste en utilizar un **Connection Pool (Piscina de Conexiones)** gestionado mediante la interfaz estándar `javax.sql.DataSource`:

> 💡 **El Estándar HikariCP: Rendimiento Extremo**
>
> **HikariCP** es la biblioteca de pool de conexiones más rápida y ligera del ecosistema Java, adoptada por defecto en frameworks como Spring Boot. Mantiene un conjunto de conexiones físicas permanentemente abiertas con el servidor. Cuando la aplicación invoca `dataSource.getConnection()`, HikariCP le entrega una conexión disponible en menos de **0,05 milisegundos**. Cuando la aplicación invoca `conn.close()`, *el proxy de HikariCP intercepta la llamada* y, en lugar de destruir el socket TCP, devuelve la conexión a la piscina para que otro hilo concurrente la reutilice inmediatamente.

```
// Configuración recomendada de HikariCP en Java 21
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/dam2_db");
config.setUsername("dam2_user");
config.setPassword("dam2_pass");

// Parámetros de ajuste profesional del pool
config.setMaximumPoolSize(10);          // Máximo 10 conexiones físicas en el servidor
config.setMinimumIdle(5);               // Mantener al menos 5 conexiones calientes preparadas
config.setConnectionTimeout(30000);     // Espera máxima de 30s antes de lanzar timeout
config.setIdleTimeout(600000);          // Cerrar conexiones inactivas tras 10 minutos
config.setLeakDetectionThreshold(5000); // Alertar si un hilo retiene una conexión más de 5s sin cerrarla

HikariDataSource dataSource = new HikariDataSource(config);
```

> 💻 **Actividad 2.7 del cuaderno de aula**
>
> **Configuración de Pool HikariCP y Benchmark Concurrente:** Configura un `HikariDataSource` en Java 21 y desarrolla un test comparativo de rendimiento que mida la velocidad de 100 consultas ejecutadas con `DriverManager` frente a `HikariCP`.
>
> 📂 Enlace directo: Ver enunciado de la Actividad 2.7

4.2. Patrones Singleton y DTO con Java Records

4.2.1. ¿Por qué necesitamos patrones?

En el Bloque 1 has hecho programas que funcionan, pero fíjate en **cómo** están hechos: el `main` abre la conexión, escribe el SQL, recorre el `ResultSet` y muestra los datos. Todo en el mismo sitio. En un ejercicio de 80 líneas no pasa nada, pero en una aplicación real con cientos de clases aparecen tres problemas:

- **El SQL está repartido por todas partes.** Si mañana cambia el nombre de una columna, hay que buscarla en todo el proyecto.
- **Cada clase abre sus propias conexiones.** MySQL tiene un límite de conexiones y la aplicación acaba fallando con el error `1040: Too many connections`.
- **No se puede probar nada sin MySQL arrancado**, porque la lógica del programa y el acceso a la base de datos están pegados.

> 💡 **¿Qué es un patrón de diseño?**
>
> Es una **solución ya probada a un problema que se repite** en muchos programas. No es una librería ni algo que se descarga: es una *forma de organizar las clases* que los equipos de desarrollo conocen por su nombre. Si en una empresa alguien dice «hazle un DAO a la tabla clientes», todo el mundo sabe qué clases hay que crear.

Mira el mismo programa (mostrar el producto 3) escrito de las dos formas:

× Antes: todo mezclado en el main

```
public static void main(String[] args) {
    String sql = "SELECT id_producto, nombre, precio,
stock "
               + "FROM productos WHERE id_producto = ?";
    try (Connection conn =
DriverManager.getConnection(URL, USER, PASS);
         PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setLong(1, 3);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                System.out.println(rs.getString("nombre")
                        + " - " +
rs.getBigDecimal("precio"));
            }
        }
    } catch (SQLException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
```

✓ Después: cada cosa en su sitio

```
public static void main(String[] args) {
    ProductoDAO dao = new ProductoDAOJdbc(
            DataSourceProvider.get());

    dao.findById(3L).ifPresent(p ->
        System.out.println(p.nombre() + " - " +
p.precio()));
}

// Ni SQL, ni Connection, ni ResultSet.
// El main solo dice QUÉ quiere, no CÓMO se consigue.
```

En la versión de la derecha aparecen los tres patrones de este bloque. Cada uno se encarga de una sola cosa:

Singleton

¿De dónde salen las conexiones?

`DataSourceProvider` guarda **un único pool** de conexiones para todo el programa.

DTO

¿Cómo viajan los datos?

`ProductoDTO` es una «caja» con los datos de **una fila**: id, nombre, precio y stock.

DAO

¿Dónde está el SQL?

`ProductoDAO` es **el único sitio** del programa que contiene SQL de la tabla `productos`.

💡 Una analogía para no perderse: el restaurante

EN EL RESTAURANTE...	EN NUESTRA APLICACIÓN...	PATRÓN
El cliente pide «el plato número 3» sin entrar en la cocina.	El `main` llama a `dao.findById(3L)`.	—
La carta dice *qué* platos se pueden pedir, no cómo se cocinan.	La interfaz `ProductoDAO`.	DAO
El cocinero sabe las recetas y es el único que entra en la despensa.	`ProductoDAOJdbc`: sabe el SQL y es el único que habla con MySQL.	DAO
El plato llega en una bandeja con lo que pediste, nada más.	Un `ProductoDTO` con los datos de la fila.	DTO
Para ir a la despensa se usan carros que se cogen y se devuelven.	Las conexiones del pool HikariCP.	Pool (4.1)
Hay **un único** cuarto de carros para todo el restaurante.	`DataSourceProvider` da siempre el mismo pool.	Singleton
La despensa.	La base de datos MySQL.	—

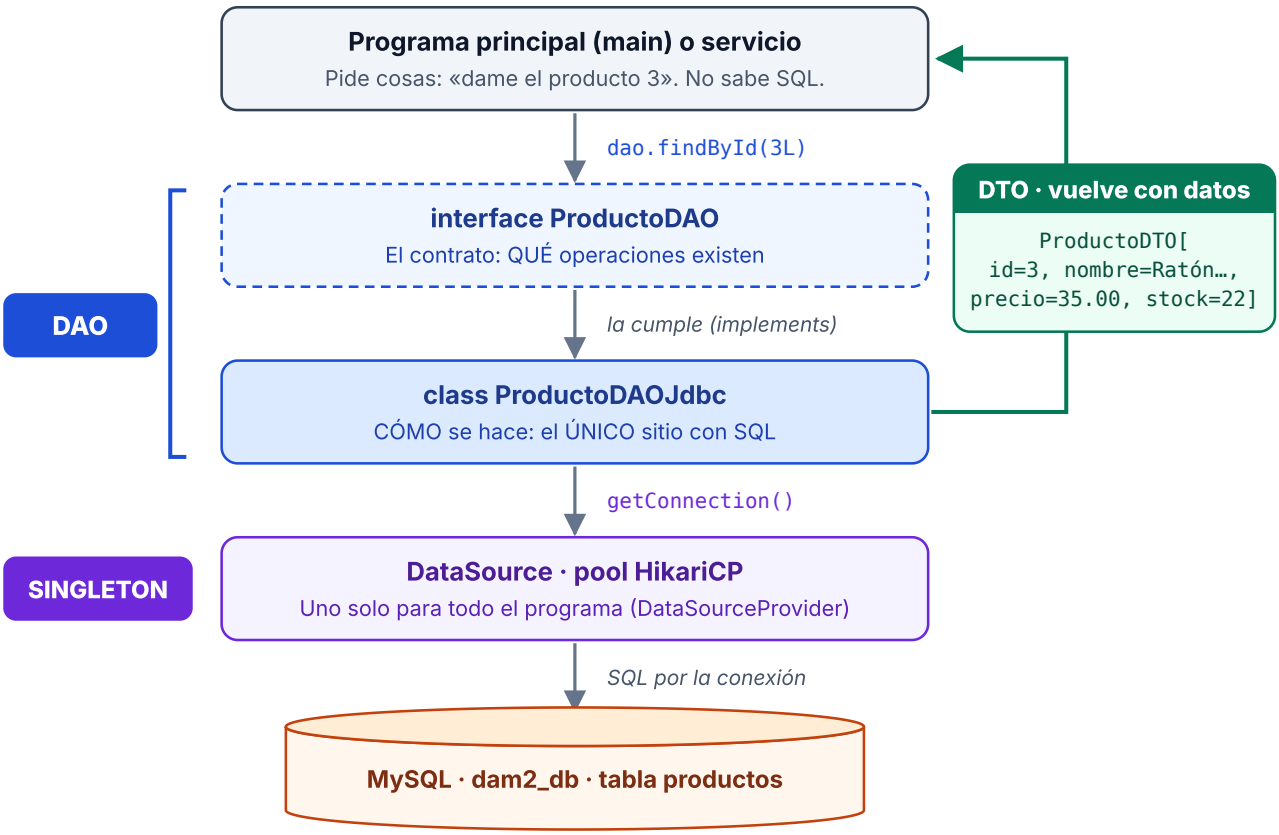


Figura 4.1. Dónde vive cada patrón. Las peticiones bajan (flechas grises) y los datos suben dentro de un DTO (flecha verde). Solo la caja «ProductoDAOJdbc» contiene SQL.

> 📂 **Mapa conceptual del bloque**
>
> Tienes todos los conceptos de este apartado y del siguiente resumidos en una sola hoja, con una segunda página en blanco para completar como repaso: **AD-UT2-Mapa-Conceptual-Patrones-Persistencia**.

4.2.2. Patrón Singleton: un único pool para toda la aplicación

EN UNA FRASE

Un **Singleton** es una clase de la que solo puede existir **un objeto**, y todo el programa usa ese mismo objeto.

¿Qué problema resuelve? En el apartado 4.1 creaste un `HikariDataSource`. Cada pool, al crearse, abre varias conexiones con MySQL y las mantiene abiertas. Si cada DAO creara su propio pool, el número de conexiones se multiplicaría sin control:

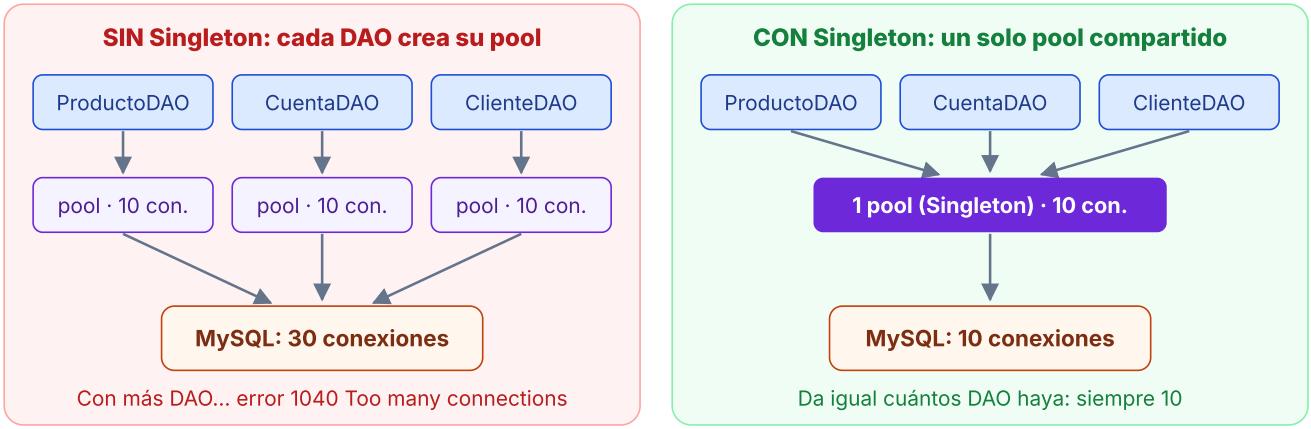


Figura 4.2. El Singleton garantiza que todos los DAO comparten las mismas conexiones.

¿Cómo se construye? Un Singleton siempre tiene **tres ingredientes**. Si los reconoces, reconocerás el patrón en cualquier código:

① **Constructor** `private`: nadie puede hacer `new DataSourceProvider()` desde fuera.

② **Un atributo** `static final` que guarda la única instancia. Al ser `static` pertenece a la clase, no a cada objeto, y por eso solo hay uno; al ser `final`, nadie puede sustituirlo por otro.

③ **Un método** `static` **público** para pedir esa instancia. Es la única puerta de entrada.

En esta unidad lo escribimos así (es el mismo Singleton que usarás en la práctica del Instituto y en el proyecto del banco):

```
public final class DataSourceProvider {

    // ① Constructor privado: nadie puede hacer "new DataSourceProvider()".
    private DataSourceProvider() { }

    // Clase interna que guarda la instancia (Singleton de Bill Pugh)
    private static class Holder {
        // ② La única instancia: static (de la clase) y final (no cambia)
        private static final DataSource INSTANCE = crearPool();
    }

    // ③ Puerta de entrada: todo el programa pide el pool por aquí.
    public static DataSource get() {
        return Holder.INSTANCE;
    }

    // Crea el pool con la configuración del apartado 4.1
    private static DataSource crearPool() {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl("jdbc:mysql://localhost:3306/dam2_db"
                + "?useSSL=false&allowPublicKeyRetrieval=true");
        cfg.setUsername("dam2_user");
```

```
        cfg.setPassword("dam2_pass");
        cfg.setMaximumPoolSize(10);
        return new HikariDataSource(cfg);
    }
}
```

¿Y para qué sirve la clase interna `Holder`? Java no carga una clase hasta que alguien la usa por primera vez. Por eso el pool no se crea al arrancar el programa, sino la primera vez que se llama a `get()`. Además, Java garantiza que esa creación ocurre **una sola vez aunque varios hilos llamen a `get()` a la vez**, sin que tengamos que escribir nada más. A esta forma de escribir el Singleton se la conoce como *Singleton de Bill Pugh* o *Holder*.

Compruébalo tú mismo: si pides el pool dos veces, recibes **el mismo objeto**.

```
DataSource ds1 = DataSourceProvider.get();
DataSource ds2 = DataSourceProvider.get();
System.out.println(ds1 == ds2);   // true → es el mismo pool, no se ha creado otro
```

> 💡 **«También lo he visto escrito con getInstance()»**
>
> Es la versión clásica del Singleton, que verás en muchos libros y webs. Tiene los mismos tres ingredientes, pero el objeto se crea dentro de un `if`:
>
> ```
> private static DataSourceProvider instancia;          // ② (todavía vacía)
> private DataSourceProvider() { ... }                  // ①
> public static synchronized DataSourceProvider getInstance() {   // ③
>     if (instancia == null) {
>         instancia = new DataSourceProvider();         // se crea solo la primera vez
>     }
>     return instancia;
> }
> ```
>
> Sin `synchronized` **no es segura**: si dos hilos entran a la vez en el `if`, los dos ven `null` y se crean dos objetos. Con `synchronized` sí es segura, pero cada llamada tiene que esperar su turno. La versión `Holder` consigue lo mismo sin `if` y sin `synchronized`; por eso es la que usamos.

> ⚠️ **Errores típicos con el Singleton**
>
> - Escribir `new HikariDataSource(config)` dentro de un DAO o, peor, dentro de cada método: se crea un pool nuevo en cada llamada.
> - Dejar el constructor `public`: entonces ya no es un Singleton, porque cualquiera puede crear otro objeto.
> - Escribir el Singleton clásico con `if (instancia == null)` y olvidar el `synchronized`: con varios hilos pueden crearse dos pools.

4.2.3. Patrón DTO: los datos que viajan entre capas

EN UNA FRASE

Un **DTO** (*Data Transfer Object*) es un objeto que solo sirve para **llevar datos** de un sitio a otro. No tiene lógica, no tiene SQL y no sabe nada de la base de datos.

¿Qué problema resuelve? Un método de Java solo puede devolver *una* cosa. Si el DAO tiene que devolver los cuatro datos de un producto, ¿qué devuelve?

- ¿Cuatro variables sueltas? No se puede.
- ¿El `ResultSet`? Tampoco: el `try-with-resources` lo cierra al salir del método (apartado 2.3) y fuera ya no sirve.
- La solución es **meter los datos de la fila en un objeto** y devolver ese objeto. Ese objeto es el DTO.

Cada columna de la consulta se convierte en un componente del DTO. Esto es justo lo que pide el criterio CE2.g: *definir los objetos destinados a almacenar el resultado de las consultas*.

COLUMNA EN MYSQL	TIPO SQL	COMPONENTE DEL DTO	TIPO JAVA	SE LEE CON
`id_producto`	BIGINT AUTO_INCREMENT	`id`	`Long`	`rs.getLong("id_producto")`
`nombre`	VARCHAR	`nombre`	`String`	`rs.getString("nombre")`
`precio`	DECIMAL(10,2)	`precio`	`BigDecimal`	`rs.getBigDecimal("precio")`
`stock`	INT	`stock`	`Integer`	`rs.getInt("stock")`

Hay dos formas de escribir un DTO en Java. La tradicional es un **POJO** (*Plain Old Java Object*): una clase normal con atributos privados, constructor, getters, setters, `equals()`, `hashCode()` y `toString()`. Desde Java 16 existe una forma mucho más corta, el `record`:

POJO tradicional (unas 50 líneas)

```
public class ProductoDTO {
    private Long id;
    private String nombre;
    private BigDecimal precio;
    private Integer stock;

    public ProductoDTO(Long id, String nombre, ...) { ...
}
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    // ... getters y setters de los otros 3 datos
    // ... equals(), hashCode() y toString()
}
```

Record (Java 16+): una línea

```
public record ProductoDTO(
        Long id,
        String nombre,
        BigDecimal precio,
        Integer stock) { }

// Java genera solo: atributos, constructor,
// p.nombre(), equals(), hashCode() y toString().
// No hay setters: el record es inmutable.
```

Los dos sirven como DTO; en esta unidad usamos **records**. El POJO lo seguirás encontrando en mucho código y volverá a aparecer en la UT3, porque las entidades de JPA/Hibernate no pueden ser records. El record completo de la Actividad 2.8 añade un constructor sin `id`:

```
public record ProductoDTO(Long id, String nombre, BigDecimal precio, Integer stock) {

    // Constructor extra para productos NUEVOS: todavía no tienen id,
    // porque el id lo pone MySQL (AUTO_INCREMENT) al hacer el INSERT.
    public ProductoDTO(String nombre, BigDecimal precio, Integer stock) {
        this(null, nombre, precio, stock);
    }
}
```

TÚ ESCRIBES...	JAVA GENERA AUTOMÁTICAMENTE...
`record ProductoDTO(Long id, String nombre, BigDecimal precio, Integer stock)`	Los 4 atributos `private final`
	El constructor con los 4 datos: `new ProductoDTO(3L, "Ratón", precio, 22)`
	Un método para leer cada dato, **sin «get»**: `p.nombre()`, `p.precio()`
	`equals()`, `hashCode()` y `toString()`

Dos detalles que suelen confundir:

- **¿Por qué `Long` e `Integer` y no `long` e `int`?** Porque los tipos envoltorio pueden valer `null`: un producto nuevo aún no tiene `id`, y una columna de MySQL puede estar a NULL (recuerda la trampa del apartado 3.3).
- **Un record es inmutable: no tiene setters.** Para «cambiar» un dato se crea un DTO nuevo con los valores nuevos:

```
ProductoDTO p = dao.findById(3L).orElseThrow();
// p.setPrecio(...)   ✗ no compila: los records no tienen setters
ProductoDTO rebajado = new ProductoDTO(p.id(), p.nombre(), new BigDecimal("29.99"), p.stock());
dao.update(rebajado);   // ✓ se guarda el DTO nuevo
```

> ⚠️ **Errores típicos con el DTO**
>
> - Escribir `p.getNombre()`: en un record el método se llama igual que el dato, `p.nombre()`.
> - Meter dentro del DTO código que abre conexiones o ejecuta SQL. El DTO solo transporta datos; el SQL va en el DAO.
> - Usar `int` para el `id` y no poder representar «todavía no tiene id».

> 🔎 **Para saber más: Entidad frente a DTO**
>
> En esta unidad, `ProductoDTO` tiene exactamente las columnas de la tabla. En aplicaciones más grandes se separan dos clases: la **entidad**, que es una copia exacta de la tabla (incluidos datos internos como un `password_hash`), y el **DTO**, que lleva solo lo que necesita quien lo recibe. Un ejemplo: la entidad `Alumno` guarda la fecha de nacimiento, pero el `AlumnoDTO` lleva directamente la edad. Lo trabajarás en la práctica del Instituto y lo retomaremos en la UT3 (mapeo objeto-relacional).

4.3. Patrón DAO Desacoplado con Inyección de DataSource

4.3.1. La idea: un único sitio para el SQL

EN UNA FRASE

Un **DAO** (*Data Access Object*) es la clase que contiene **todo el SQL de una tabla**. El resto del programa le pide cosas («guarda este producto», «dame el producto 3») sin saber cómo las hace.

La regla práctica es sencilla: **una tabla, un DAO**. La tabla `productos` tiene su `ProductoDAO`; la tabla `cuentas`, su `CuentaDAO`. Y cada DAO tiene **dos piezas**:

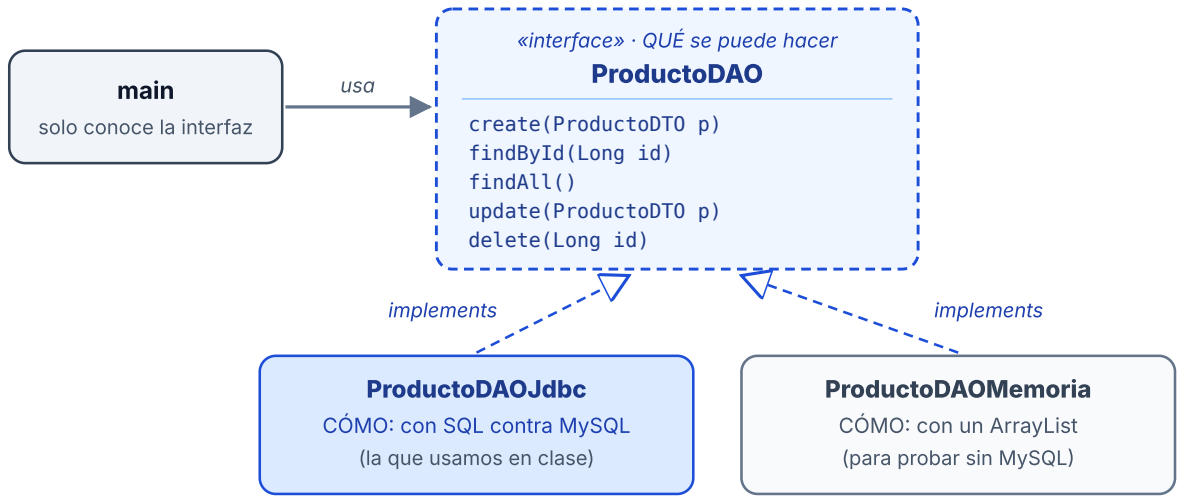


Figura 4.3. La interfaz es el contrato; la implementación es el código real. Como el `main` solo conoce la interfaz, funciona igual con cualquiera de las dos implementaciones.

Piensa en un **enchufe**: la pared tiene un enchufe con una forma fija (la interfaz) y puedes conectar una lámpara o un cargador (las implementaciones). Si mañana la base de datos pasa de MySQL a PostgreSQL, solo se escribe otra implementación; el `main` y el resto del programa no se tocan. A esto se le llama **desacoplar**.

Las operaciones de un DAO son siempre las del CRUD que ya conoces del apartado 3.2, ahora metidas en métodos con nombre:

MÉTODO	QUÉ HACE	SQL QUE LLEVA DENTRO	DEVUELVE
`create(ProductoDTO p)`	Guarda un producto nuevo	`INSERT`	El `ProductoDTO` con el id que ha generado MySQL
`findById(Long id)`	Busca un producto por su clave	`SELECT … WHERE id_producto = ?`	`Optional<ProductoDTO>` (puede no existir)
`findAll()`	Devuelve todos los productos	`SELECT … ORDER BY`	`List<ProductoDTO>` (vacía si no hay, nunca `null`)
`update(ProductoDTO p)`	Modifica un producto	`UPDATE`	`true` si ha cambiado alguna fila
`delete(Long id)`	Borra un producto	`DELETE`	`true` si existía y se ha borrado

4.3.2. Paso 1: la interfaz (el contrato)

```
public interface ProductoDAO {
    ProductoDTO create(ProductoDTO prod);          // INSERT
    Optional<ProductoDTO> findById(Long id);        // SELECT por clave
    List<ProductoDTO> findAll();                    // SELECT de todos
    boolean update(ProductoDTO prod);              // UPDATE
    boolean delete(Long id);                       // DELETE
}
```

Fíjate en lo que **no** aparece: ni `Connection`, ni `ResultSet`, ni `SQLException`. Nada de `java.sql`. Por eso quien usa el DAO no necesita saber que detrás hay JDBC.

4.3.3. Paso 2: una interfaz genérica para todos los DAO

Si la aplicación tiene diez tablas, habrá diez DAO, y los diez tendrán las mismas cinco operaciones del CRUD. Para no repetir el contrato diez veces se escribe **una sola vez** con genéricos, igual que `List<String>` o `List<Integer>`:

```
// T  = el tipo de objeto que se guarda (ProductoDTO, Alumno, CuentaDTO...)
// ID = el tipo de su clave primaria  (Long, String...)
public interface GenericDAO<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
    T save(T entity);               // INSERT
    boolean update(T entity);       // UPDATE
```

```
    boolean deleteById(ID id);       // DELETE
}

// Cada DAO concreto solo dice QUÉ tipos usa (y añade sus métodos propios, si los tiene)
public interface AlumnoDAO extends GenericDAO<Alumno, String> { }
public interface CuentaDAO extends GenericDAO<CuentaDTO, String> {
    Optional<CuentaDTO> findByIdForUpdate(Connection conn, String iban);
}
```

Es exactamente el mismo CRUD que en `ProductoDAO`; solo cambian los nombres de algunos métodos (`save` en lugar de `create`, `deleteById` en lugar de `delete`). En la Actividad 2.8 escribirás `ProductoDAO` directamente, sin genéricos, para centrarte en la idea; en la práctica del Instituto usarás `GenericDAO<T, ID>`.

4.3.4. Paso 3: la implementación JDBC

Aquí sí está el SQL, y es el mismo que escribiste en la Actividad 2.2. Los números de los comentarios se explican debajo del código:

```
public class ProductoDAOJdbc implements ProductoDAO {

    private final DataSource dataSource;                        // ①

    public ProductoDAOJdbc(DataSource dataSource) {             // ①
        this.dataSource = dataSource;
    }

    @Override
    public Optional<ProductoDTO> findById(Long id) {
        String sql = "SELECT id_producto, nombre, precio, stock FROM productos WHERE id_producto = ?";
        try (Connection conn = dataSource.getConnection();       // ②
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(mapearFila(rs));          // ③ ⑤
                }
                return Optional.empty();                         // ⑤
            }
        } catch (SQLException e) {
            throw new RuntimeException("Error al buscar el producto " + id, e);   // ④
        }
    }

    @Override
    public List<ProductoDTO> findAll() {
        String sql = "SELECT id_producto, nombre, precio, stock FROM productos ORDER BY id_producto";
        List<ProductoDTO> lista = new ArrayList<>();
        try (Connection conn = dataSource.getConnection();       // ②
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                lista.add(mapearFila(rs));                       // ③
            }
```

```
            return lista;
        } catch (SQLException e) {
            throw new RuntimeException("Error al listar productos", e);          // ④
        }
    }

    // create, update y delete siguen el mismo esquema (ver Actividad 2.8)

    // ③ Convierte la fila ACTUAL del ResultSet en un DTO. Un único sitio para esto.
    private ProductoDTO mapearFila(ResultSet rs) throws SQLException {
        return new ProductoDTO(
                rs.getLong("id_producto"),
                rs.getString("nombre"),
                rs.getBigDecimal("precio"),
                rs.getInt("stock"));
    }
}
```

① **Recibe el** `DataSource` **por el constructor.** El DAO no crea el pool ni conoce el usuario y la contraseña de MySQL: se lo dan hecho (lo explicamos en 4.3.5).

② **Cada método pide una conexión y la devuelve.** Gracias al `try-with-resources`, al terminar el método la conexión vuelve al pool (CE2.i: eliminar los objetos cuando terminan su función).

③ `mapearFila` **convierte una fila en un DTO.** Lo usan `findById` y `findAll`, así la conversión está escrita una sola vez.

④ **La** `SQLException` **no sale del DAO.** Se envuelve en una `RuntimeException` con un mensaje claro. Así la interfaz no tiene que declarar `throws SQLException` y el `main` no depende de JDBC. En la práctica del Instituto crearás tu propia excepción, `DaoException extends RuntimeException`, que además guarda el código de error de MySQL.

⑤ `findById` **devuelve** `Optional`**, nunca** `null`**.** Un `Optional` es una caja que puede estar llena o vacía, y obliga a quien llama a decidir qué hacer si el producto no existe.

4.3.5. ¿Qué significa «inyectar» el DataSource?

Inyectar una dependencia significa **dársela a la clase desde fuera** (normalmente por el constructor), en lugar de que la clase la cree ella misma. Al cocinero le dan la llave de la despensa cuando entra a trabajar; no se fabrica una llave propia.

× El DAO se crea sus conexiones

```
public class ProductoDAOJdbc implements ProductoDAO {
    public Optional<ProductoDTO> findById(Long id) {
        try (Connection conn =
                DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/dam2_db",
                "dam2_user", "dam2_pass")) {
            ...
// URL y contraseña repetidas en cada DAO,
// conexión nueva y lenta en cada llamada,
// imposible usar otra BD para hacer pruebas.
```

✓ Se lo dan por el constructor

```
public class ProductoDAOJdbc implements ProductoDAO {
    private final DataSource dataSource;

    public ProductoDAOJdbc(DataSource dataSource) {
        this.dataSource = dataSource;
    }
    ...
// El DAO no sabe de dónde vienen las conexiones.
// Todos los DAO comparten el pool del Singleton.
// En pruebas se le puede pasar otro DataSource.
```

> 💡 **Y en el proyecto del banco (apartado 5.1)...**
>
> Allí los métodos del DAO reciben una `Connection` como parámetro, por ejemplo `cuentaDAO.actualizarSaldo(conn, iban, saldo)`. Es la misma idea de inyectar: como la transferencia debe ser una única transacción, es el servicio quien abre la conexión, hace `commit()` o `rollback()`, y se la va pasando a los DAO.

4.3.6. Todo junto: qué pasa en una llamada

```
public class AppProductos {
    public static void main(String[] args) {
        // El Singleton da el pool y se lo inyectamos al DAO
        ProductoDAO dao = new ProductoDAOJdbc(DataSourceProvider.get());

        dao.findById(3L).ifPresentOrElse(
            p -> System.out.println(p.nombre() + " cuesta " + p.precio() + " €"),
            () -> System.out.println("No existe el producto 3"));
    }
}
```

Esto es lo que ocurre, paso a paso, cuando se ejecuta `dao.findById(3L)`:

1. **El** `main` llama a `findById(3L)`. Solo conoce la interfaz `ProductoDAO`.
2. **El DAO** pide una conexión: `dataSource.getConnection()`.
3. **El pool** (el único, gracias al Singleton) le presta una conexión que ya estaba abierta.
4. **El DAO** prepara el `SELECT` con `PreparedStatement`, pone el `3` en el `?` y lo ejecuta en MySQL.
5. **MySQL** devuelve la fila en un `ResultSet`.
6. `mapearFila` copia los datos de la fila en un `ProductoDTO`.

7 **El** `try-with-resources` cierra el `ResultSet` y el `PreparedStatement`, y devuelve la conexión al pool.

8 **El** `main` recibe un `Optional<ProductoDTO>` y muestra «Ratón Inalámbrico Ergonómico cuesta 35.00 €».

> 🏭 **En la Actividad 2.8**
>
> En el programa de prueba de la actividad el pool se crea directamente en el `main` con `try (HikariDataSource ds = new HikariDataSource(config))`. Es válido porque es un programa pequeño con un solo DAO y un solo pool. En cuanto la aplicación tenga varios DAO, como en el proyecto del banco, usa `DataSourceProvider`.

4.3.7. Resumen: los tres patrones de un vistazo

PATRÓN	PREGUNTA QUE RESPONDE	IDEA EN UNA FRASE	EN NUESTRO CÓDIGO	CÓMO LO RECONOCES
Singleton	¿De dónde salen las conexiones?	Solo existe un objeto de la clase y todos usan ese.	`DataSourceProvider`	Constructor `private` + atributo `static` + método `static` de acceso.
DTO	¿Cómo viajan los datos?	Una caja con los datos de una fila, sin lógica.	`ProductoDTO`	Un `record` (o clase) con solo datos; sin SQL ni conexiones.
DAO	¿Dónde está el SQL?	Una clase por tabla que contiene todo su SQL.	`ProductoDAO` + `ProductoDAOJdbc`	Interfaz con el CRUD + clase que la implementa y recibe el `DataSource`.

> ⚠️ **La prueba del algodón**
>
> Cuando termines la Actividad 2.8, revisa tu `main`: si aparece un `import java.sql.*`, una cadena con `SELECT` o un `ResultSet`, algo se ha escapado del DAO.

💻 Actividad 2.8 del cuaderno de aula

Arquitectura Limpia DAO / DTO con HikariDataSource: Diseña una capa DAO completa para la gestión de productos, modelando entidades con Java Records, desacoplando la persistencia mediante interfaces e inyectando un `DataSource` de HikariCP.

📂 Enlace directo: Ver enunciado de la Actividad 2.8

🧠 Comprueba que sabes (Bloque 2)

1. ¿Por qué el método `conn.close()` en una conexión obtenida de HikariCP no cierra el socket físico de red?
2. Si llamas dos veces a `DataSourceProvider.get()`, ¿se crean dos pools? ¿Qué parte del código lo impide?
3. Un compañero escribe `p.setPrecio(new BigDecimal("10"))` sobre un `ProductoDTO` y no compila. ¿Por qué? ¿Cómo cambiarías el precio?
4. ¿Por qué `findById` devuelve `Optional<ProductoDTO>` y no directamente un `ProductoDTO`?
5. Si la base de datos pasara de MySQL a PostgreSQL, ¿qué clases tendrías que cambiar? ¿Cuáles no tocarías?
6. ¿Por qué es una buena práctica de diseño inyectar `javax.sql.DataSource` en el constructor del DAO en lugar de llamar a `DriverManager.getConnection()` dentro de cada método?

✅ Ya puedes hacer

Una vez dominados los pools de conexiones y la estructuración en capas desacopladas con Singleton, Records y DAOs, puedes resolver:

- **AD-UT2-B2-Ejercicios-Patrones-Pool**: Detección de fugas de conexión con HikariCP y capa de persistencia completa DAO/DTO con transacciones inyectadas.
- **PRAC-AD-UT2-Instituto-DAO-DTO-MySQL**: práctica de afianzamiento con los tres patrones (Singleton, DTO y DAO) sobre la base de datos del instituto, incluida la diferencia entre entidad y DTO.

5. Integración: Backend Bancario Transaccional con DAO y Pool

Para culminar la unidad, todos los conceptos aprendidos (conexión mediante pool, modelado DTO con Records, interfaces DAO desacopladas y transacciones ACID en MySQL) se coordinan armónicamente en un caso de uso real de misión crítica: un **Servicio Bancario Transaccional**.

5.1. Backend Bancario Transaccional con DAO y Pool

El diseño del sistema bancario respeta la separación estricta de responsabilidades en cuatro capas:

1. **Capa de Infraestructura (Pool):** Clase `DataSourceProvider` (Singleton) que inicializa y expone el `HikariDataSource` conectado a MySQL.
2. **Capa de Modelo (DTOs):** Records inmutables `CuentaDTO(String iban, String titular, BigDecimal saldo, String estado)` y `MovimientoDTO(Long id, String ibanOrigen, String ibanDestino, BigDecimal cantidad, String tipo, LocalDateTime fecha)`.
3. **Capa de Persistencia (DAOs):** Interfaces `CuentaDAO` y `MovimientoDAO` con implementaciones JDBC que reciben conexiones o DataSources y ejecutan sentencias con `PreparedStatement` y `CallableStatement`.
4. **Capa de Servicio de Negocio (`TransferenciaService`):** Es la encargada de abrir la transacción manual, obtener la conexión del pool, pasarla a los métodos DAO correspondientes, verificar las reglas de negocio (importe positivo, cuentas distintas, cuenta activa, saldo suficiente) y ejecutar el `commit()` o `rollback()`. Los DAO **nunca** abren ni cierran la transacción: reciben la `Connection` del servicio.

Para que la transferencia sea realmente atómica y segura frente a concurrencia hay dos detalles clave. Primero, las cuentas se leen con `SELECT ... FOR UPDATE` (método `findByIdForUpdate`), que en InnoDB bloquea esas filas hasta el `commit()` o `rollback()`; sin este bloqueo, dos transferencias simultáneas podrían leer el mismo saldo y una pisaría a la otra (*lost update*). Segundo, las cuentas se bloquean **siempre en el mismo orden** (por ejemplo, el IBAN menor primero); si cada transferencia bloquea en un orden distinto, dos transferencias cruzadas (A→B y B→A) pueden provocar un *deadlock* (error 1213, véase el apartado 6).

```
// Coordinación transaccional en la Capa de Servicio
public class TransferenciaService {
    private final DataSource dataSource;
    private final CuentaDAO cuentaDAO;
    private final MovimientoDAO movimientoDAO;

    public TransferenciaService(DataSource ds, CuentaDAO cDao, MovimientoDAO mDao) {
        this.dataSource = ds;
        this.cuentaDAO = cDao;
        this.movimientoDAO = mDao;
    }

    public void transferir(String ibanOrigen, String ibanDestino, BigDecimal importe) throws SQLException {
        // 0. Reglas que no necesitan la base de datos: fallar pronto, sin abrir conexión
        if (importe == null || importe.signum() <= 0) {
            throw new IllegalArgumentException("El importe debe ser positivo");
        }
        if (ibanOrigen.equals(ibanDestino)) {
```

```
            throw new IllegalArgumentException("Origen y destino no pueden ser la misma cuenta");
        }

        try (Connection conn = dataSource.getConnection()) {
            conn.setAutoCommit(false); // Inicio de la transacción ACID
            try {
                // 1. Bloquear AMBAS cuentas (SELECT ... FOR UPDATE) siempre en el mismo orden
                //    para evitar lost updates y deadlocks entre transferencias cruzadas
                boolean origenPrimero = ibanOrigen.compareTo(ibanDestino) < 0;
                CuentaDTO primera = cuentaDAO.findByIdForUpdate(conn, origenPrimero ? ibanOrigen :
ibanDestino)
                    .orElseThrow(() -> new IllegalArgumentException("Cuenta inexistente"));
                CuentaDTO segunda = cuentaDAO.findByIdForUpdate(conn, origenPrimero ? ibanDestino :
ibanOrigen)
                    .orElseThrow(() -> new IllegalArgumentException("Cuenta inexistente"));
                CuentaDTO origen  = origenPrimero ? primera : segunda;
                CuentaDTO destino = origenPrimero ? segunda : primera;

                // 2. Reglas de negocio con los datos ya bloqueados
                if (!"ACTIVA".equals(origen.estado()) || !"ACTIVA".equals(destino.estado())) {
                    throw new IllegalStateException("Ambas cuentas deben estar activas");
                }
                if (origen.saldo().compareTo(importe) < 0) {
                    throw new SaldoInsuficienteException("Saldo insuficiente en cuenta origen");
                }

                // 3. Ejecutar los dos débitos/créditos atómicos
                cuentaDAO.actualizarSaldo(conn, ibanOrigen, origen.saldo().subtract(importe));
                cuentaDAO.actualizarSaldo(conn, ibanDestino, destino.saldo().add(importe));

                // 4. Registrar movimiento en la auditoría
                movimientoDAO.registrar(conn, new MovimientoDTO(null, ibanOrigen, ibanDestino, importe,
"TRANSFERENCIA", LocalDateTime.now()));

                // 5. Confirmación definitiva de la transacción
                conn.commit();
            } catch (SQLException | RuntimeException e) {
                conn.rollback(); // Deshacer todas las modificaciones ante cualquier fallo
                throw e;         // y propagar el error original a quien llamó
            } finally {
                conn.setAutoCommit(true);
            }
        }
    }
}
```

✅ **Ya puedes hacer**

Has cubierto todos los contenidos curriculares y técnicos de la unidad. Ya estás plenamente capacitado para abordar y entregar el proyecto final de evaluación:

- **PRAC-AD-UT2-Banco-MySQL-DAO-Transaccional**: Proyecto Capstone: Sistema Bancario Transaccional Seguro con MySQL, DAO y Connection Pool (con Rúbrica Analítica de 10 Puntos).

6. Visión de Conjunto y Buenas Prácticas de Ingeniería

A continuación se sintetizan las pautas de ingeniería de software que diferencian el código novato de una implementación profesional en producción:

ÁREA	PRÁCTICA INACEPTABLE (JUNIOR / NOVATO)	ESTÁNDAR PROFESIONAL DE EMPRESA (SENIOR)
Construcción SQL	Concatenar cadenas con `+` en objetos `Statement` exponiendo la base a SQL Injection.	Uso estricto de `PreparedStatement` con parámetros tipados `?`.
Ciclo de Conexión	Llamar a `DriverManager.getConnection()` en cada endpoint o petición web.	Pool de conexiones `HikariCP` inyectado a través de `DataSource`.
Liberación de Recursos	Confiar en el recolector de basura o cerrar manualmente en bloques de código desprotegidos.	Uso determinista de `try-with-resources` para toda conexión, sentencia y cursor.
Manejo de Transacciones	Dejar `autoCommit=true` y realizar múltiples operaciones dependientes por separado.	Control manual con `setAutoCommit(false)`, `commit()` y `rollback()` en bloque catch, sobre tablas **InnoDB** y con `SELECT ... FOR UPDATE` cuando se lee un dato que se va a modificar.
Mapeo de Datos	Dispersar columnas SQL a lo largo del código de interfaz o usar estructuras genéricas como `Map`.	Modelado fuertemente tipado e inmutable con **Java Records** y capas **DAO** desacopladas.
Credenciales	Escribir URL, usuario y contraseña directamente en el código fuente (y subirlos al repositorio).	Leerlos de un fichero de configuración o de variables de entorno, fuera del control de versiones. (En clase se dejan en el código solo por simplicidad.)
Gestión de Errores	Capturar `SQLException` y solo hacer `printStackTrace()`, o tragarse la excepción sin avisar.	Distinguir el fallo con `getErrorCode()` y `getSQLState()`, hacer `rollback()` y propagar o traducir el error a una excepción de negocio comprensible.

> ⚠️ **Catálogo de Códigos de Error Críticos en MySQL (`SQLException.getErrorCode()`)**
>
> - **1045 · Access denied for user:** Credenciales incorrectas o usuario sin permisos sobre la base de datos (`SQLState 28000`).
> - **1062 · Duplicate entry for key:** Intento de insertar una clave primaria o campo `UNIQUE` duplicado (`SQLState 23000`).
> - **1048 · Column cannot be null:** Se intentó insertar un valor nulo en una columna con restricción `NOT NULL` (`SQLState 23000`).
> - **1040 · Too many connections:** Fuga crítica de conexiones (`Connection` sin cerrar con `try-with-resources`) que agota el máximo del servidor.
> - **1213 · Deadlock found when trying to get lock:** Conflicto de concurrencia entre transacciones; MySQL aborta una de ellas y hay que reintentarla (`SQLState 40001`). Se previene bloqueando las filas siempre en el mismo orden.
> - **1452 · Cannot add or update a child row (foreign key):** Se inserta una fila que referencia una clave ajena que no existe (por ejemplo, un movimiento de una cuenta inexistente) (`SQLState 23000`).

7. Resumen y Conceptos que Debes Dominar

Al concluir esta unidad de trabajo, debes ser capaz de demostrar de forma práctica las siguientes habilidades técnicas en tu entorno de desarrollo:

- ✅ **Configurar conectividad empresarial:** Declarar dependencias Maven oficiales (`mysql-connector-j` , `HikariCP`) y conectar Java con servidores MySQL locales o remotos.
- ✅ **Escribir código SQL seguro y precompilado:** Desarrollar operaciones DML con `PreparedStatement` , previniendo radicalmente vulnerabilidades de inyección SQL y recuperando claves AUTO_INCREMENT generadas.
- ✅ **Procesar resultados y tipos nulos:** Recorrer cursores `ResultSet` , mapear fechas modernas con `java.time` y gestionar valores nulos numéricos de forma robusta con `wasNull()` y `getObject()` .
- ✅ **Gobernar transacciones atómicas:** Desactivar el autocommit, coordinar transferencias de datos entre múltiples tablas y ejecutar rollbacks sistemáticos ante excepciones.
- ✅ **Invocar lógica de servidor (procedimientos almacenados):** Preparar llamadas a procedimientos con `CallableStatement` y registrar y capturar parámetros de salida con `java.sql.Types` .
- ✅ **Optimizar con Connection Pools:** Configurar piscinas HikariCP ajustando tamaños máximos, timeouts y umbrales de detección de fugas.
- ✅ **Arquitectura desacoplada DAO/DTO:** Implementar interfaces de persistencia CRUD genéricas, inyectar DataSources y modelar entidades de transferencia mediante Java Records inmutables.

8. Glosario Técnico y Cuestionario de Repaso

📖 **Glosario de Términos Clave**

ACID:
Acrónimo de Atomicidad, Consistencia, Aislamiento (Isolation) y Durabilidad; propiedades fundamentales que garantizan transacciones fiables en motores relacionales.
CallableStatement:
Subinterfaz de JDBC especializada en la invocación de procedimientos y funciones almacenadas en el servidor de base de datos.
Connection Pool:
Piscina de conexiones físicas de red mantenidas abiertas en memoria para su reutilización inmediata por múltiples hilos de ejecución.
DAO (Data Access Object):
Patrón arquitectónico que encapsula y desacopla el acceso a los datos de la lógica de negocio de la aplicación.
DTO (Data Transfer Object):
Objeto portador de datos sin comportamiento de negocio utilizado para transportar información entre capas de software.
PreparedStatement:
Sentencia SQL precompilada en el motor que admite parámetros tipados mediante marcadores posicionales.
Savepoint:
Punto de restauración intermedio dentro de una transacción que permite realizar rollbacks parciales.

❓ **Cuestionario de Autoevaluación**

1. **¿Qué ventaja de rendimiento ofrece `PreparedStatement` frente a `Statement` al ejecutar consultas repetidas?**
   Respuesta: La sentencia se prepara una sola vez y se reutiliza cambiando solo los valores de los parámetros. Con MySQL Connector/J, si se activa `useServerPrepStmts=true` en la URL, el servidor la analiza una vez y reutiliza ese análisis en cada ejecución. Pero la ventaja principal es la seguridad: impide la inyección SQL.
2. **¿Por qué el driver JDBC devuelve 0 al invocar `rs.getInt()` sobre una columna con valor NULL?**
   Respuesta: Porque los tipos primitivos de Java no admiten el valor `null`. Debe verificarse con `rs.wasNull()` o emplear `rs.getObject(col, Integer.class)`.
3. **¿Qué ocurre con la conexión física de red al invocar `conn.close()` sobre una conexión suministrada por HikariCP?**
   Respuesta: No se destruye el socket TCP; el proxy de HikariCP intercepta la llamada y retorna la conexión a la piscina para su reutilización.
4. **¿Qué interfaz de JDBC permite conocer cuántas columnas devolvió un SELECT dinámico y cuáles son sus nombres?**
   Respuesta: `ResultSetMetaData`, obtenida mediante `rs.getMetaData()`.
5. **¿Cómo se deshacen exclusivamente las modificaciones posteriores a un punto intermedio en una transacción?**
   Respuesta: Creando un punto de restauración con `Savepoint sp = conn.setSavepoint("nombre")` y ejecutando `conn.rollback(sp)`.

Anexo: Trazabilidad Curricular Oficial

La siguiente matriz certifica la cobertura absoluta de los 11 criterios de evaluación del Resultado de Aprendizaje 2 (RA2) entre los apartados teóricos de este manual, las actividades del cuaderno de aula y las prácticas de evaluación:

CRITERIO OFICIAL	DESCRIPCIÓN OFICIAL DEL REAL DECRETO 405/2023	APARTADOS	ACTIVIDAD DE AULA	PRÁCTICA / EJERCICIO
CE2.a	Se han valorado las ventajas e inconvenientes de utilizar conectores.	2.1, 5.1	Actividad 2.1	Capstone UT2
CE2.b	Se han utilizado gestores de bases de datos embebidos e independientes.	2.2, 5.1	Actividad 2.1	Capstone UT2
CE2.c	Se ha utilizado el conector idóneo en la aplicación.	2.3, 3.1, 5.1	Actividad 2.1	Capstone UT2
CE2.d	Se ha establecido la conexión.	2.3, 3.1, 4.1, 5.1	Actividad 2.1, 2.7	Capstone UT2
CE2.e	Se ha definido la estructura de la base de datos.	3.6, 5.1	Actividad 2.6	AD-UT2-B1-Ejercicios-JDBC, Capstone
CE2.f	Se han desarrollado aplicaciones que modifican el contenido de la base de datos.	3.2, 5.1	Actividad 2.2	AD-UT2-B1-Ejercicios-JDBC, Capstone
CE2.g	Se han definido los objetos destinados a almacenar el resultado de las consultas.	3.3, 3.6, 4.2, 4.3, 5.1	Actividad 2.3, 2.6, 2.8	AD-UT2-B1, AD-UT2-B2, Capstone
CE2.h	Se han desarrollado aplicaciones que efectúan consultas.	3.3, 4.3, 5.1	Actividad 2.3, 2.8	AD-UT2-B1, AD-UT2-B2, Capstone
CE2.i	Se han eliminado los objetos una vez finalizada su función.	2.3, 3.2, 4.3, 5.1	Actividad 2.2, 2.8	AD-UT2-B1, AD-UT2-B2, Capstone
CE2.j	Se han gestionado las transacciones.	3.5, 5.1	Actividad 2.5	AD-UT2-B1-Ejercicios-JDBC, Capstone
CE2.k	Se han ejecutado procedimientos almacenados en la base de datos.	3.4, 5.1	Actividad 2.4	Capstone UT2